

Description:

Introduction

In MIDI files, timestamps are stored as integers representing the number of "ticks" since the previous event in the file.

Because these timestamps are relative to one another, most values tend to be fairly small. In order to save space, MIDI files use a special integer encoding format called a **Variable Length Quantity (VLQ)**.

A MIDI VLQ stores an integer using one or more bytes:

- The lower 7 bits of each byte contain the actual data
- The highest bit indicates whether another byte follows
 - 1 means another byte follows
 - 0 means this is the final byte

For example:

0x7F -> 01111111

Since the highest bit is 0, this is the final byte.

Removing the continuation bit leaves:

1111111 = 127

Values larger than 127 require multiple bytes.

For example:

[0x81, 0x00]

in binary becomes:

10000001 00000000

Removing the continuation bits gives:

00000001 00000000

Which is:

0000000100000000 = 128

So:

[0x81, 0x00] represents 128

Task

Your task is to implement two functions:

```
encode(n)
decode(data)
```

- `encode` should convert an integer into a MIDI VLQ byte sequence
- `decode` should decode only the first complete MIDI VLQ contained in the input. Any additional bytes after the terminating VLQ byte should be ignored.

All test values will satisfy:

$$0 \leq n \leq 0xFFFFFFFF$$

Which is the maximum value representable by a 4-byte MIDI VLQ.

Examples

Encode:

```
encode(0)    == [0x00]
encode(127)  == [0x7F]
encode(128)  == [0x81, 0x00]
encode(8192) == [0xC0, 0x00]
```

Decode:

```
decode([0x00]) == 0
decode([0x7F]) == 127
decode([0x81, 0x00]) == 128
decode([0xFF, 0xFF, 0xFF, 0x7F]) == 268435455
```

Binary

Bits

Algorithms